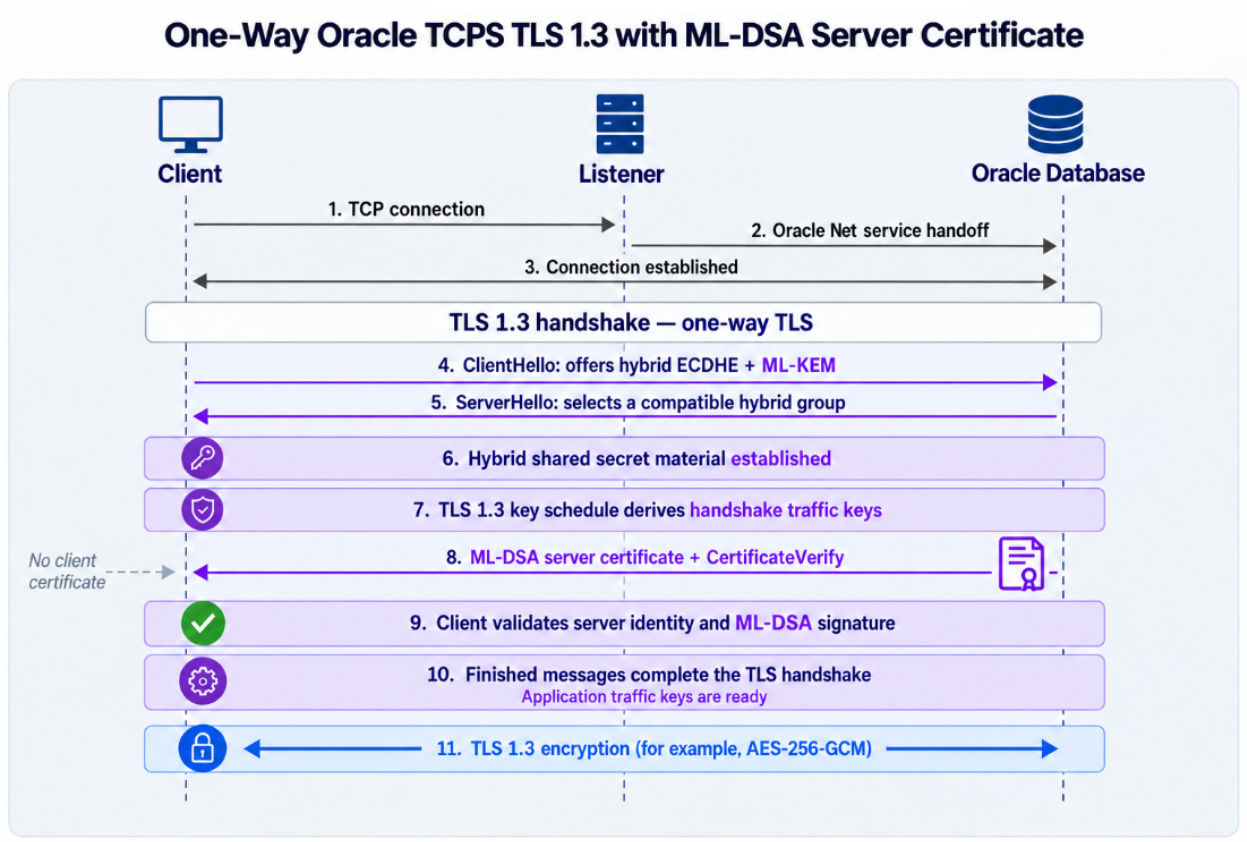
ML-DSA TLS Certificate Runbook

Oracle 19c (19.32) / 26ai — Self-Signed certificate creation and client testing

Purpose: This runbook details how to create a private ML-DSA trust hierarchy, configure a DB test listener and PDB for TCPS/TLS 1.3 (one-way), and validate connectivity from an Oracle client. For the purposes of this tutorial, an Oracle 19c DBaaS Base instance (non-RAC/Single-Instance) was used.



One-way Oracle TCPS TLS 1.3 with an ML-DSA server certificate: The server authenticates to the client using its ML-DSA certificate and signature; no client certificate is sent. Hybrid ECDHE and ML-KEM establish key material, and AES-256-GCM protects database traffic.

Database host	`dbs19c.sub0409XXXXXX250.dbsec.oraclevcn.com`
Database home	`/u01/app/oracle/product/19.0.0/dbhome_1`
Database RU	19.32

External JDK	`/u01/app/oracle/product/jdk-24`
PDB / GUID	DBS19CPDB / 569xxxxxxxxxxxxx74B8
PDB service	dbs19cpdb.`sub0409XXXXXX250.dbsec.oraclevcn.com`
TCPS endpoint	dbs19c.`sub0409XXXXXX250.dbsec.oraclevcn.com`:2484

Run as the Oracle software owner (`oracle`) on the database host.

```
export ORACLE_HOME=/u01/app/oracle/product/19.0.0/dbhome_1

python3 $ORACLE_HOME/bin/set_crypto_provider.py status

/u01/app/oracle/product/jdk-24/bin/java -version
```

Required results:

```
Current mode is: Next Generation
java version "24" (or later)
```

ML-DSA requires the next-generation cryptographic provider and JDK 24 or later. Do not switch an existing environment from legacy to next-generation provider without a separate compatibility and change-control exercise.

2. Important `orapki` JDK workaround

The database-home `orapki` shell script gives `$ORACLE_HOME/jdk` precedence over `JAVA_HOME`. The bundled Java can be too old for ML-DSA even if JDK 24 is installed separately.

For every ML-DSA `orapki` command, use this prefix:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki
```

Optionally verify JDK ML-DSA support:

```
/u01/app/oracle/product/jdk-24/bin/jshell
```

At the `jshell>` prompt:

```
import java.security.KeyPairGenerator;
KeyPairGenerator.getInstance("ML-DSA-65");
/exit
```

3. Create protected wallet directories

```
mkdir -p /u01/app/oracle/wallets/pqc-root
mkdir -p /u01/app/oracle/wallets/pqc-server
mkdir -p /u01/app/oracle/wallets/pqc-client

chmod 700 /u01/app/oracle/wallets/pqc-root
chmod 700 /u01/app/oracle/wallets/pqc-server
chmod 700 /u01/app/oracle/wallets/pqc-client
umask 077
```

Do not use or overwrite the DBaaS-managed wallet directory:

```
/opt/oracle/dcs/commonstore/tcps_wallet
```

4. Create the ML-DSA root CA wallet

Create a password-protected auto-login wallet:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet create \
    -wallet /u01/app/oracle/wallets/pqc-root \
    -auto_login
```

Create the ML-DSA root certificate:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet add \
    -wallet /u01/app/oracle/wallets/pqc-root \
    -dn "CN=PQC Root CA" \
    -asym_alg ML-DSA-65 \
    -sign_alg mldsa65 \
    -self_signed \
    -validity 3650
```

Verify it:

```
env -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  orapki wallet display \
    -wallet /u01/app/oracle/wallets/pqc-root \
    -complete
```

Export its root (public) certificate:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet export \
    -wallet /u01/app/oracle/wallets/pqc-root \
    -dn "CN=PQC Root CA" \
    -cert /u01/app/oracle/wallets/pqc-root/pqc_root_ca_cert.pem
```

Verify:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet display \
    -wallet /u01/app/oracle/wallets/pqc-root \
    -complete
```

Expected root certificate:

```
Subject: CN=PQC Root CA
Issuer:  CN=PQC Root CA
```

5. Create and sign the ML-DSA server certificate

Create the server wallet:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet create \
    -wallet /u01/app/oracle/wallets/pqc-server \
    -auto_login
```

Generate the server key and certificate request. Replace the CN with the DB server FQDN for the DBaaS system.

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet add \
    -wallet /u01/app/oracle/wallets/pqc-server \
    -dn "CN=dbs19c.sub0409XXXXXX250.dbsec.oraclevcn.com" \
    -asym_alg ML-DSA-65
```

Export the certificate request:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet export \
    -wallet /u01/app/oracle/wallets/pqc-server \
    -dn "CN=dbs19c.sub0409XXXXXX250.dbsec.oraclevcn.com" \
    -request /u01/app/oracle/wallets/pqc-server/server_mldsa_req.pem
```

Sign it with the root CA:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki cert create \
    -wallet /u01/app/oracle/wallets/pqc-root \
    -request /u01/app/oracle/wallets/pqc-server/server_mldsa_req.pem \
    -cert /u01/app/oracle/wallets/pqc-server/server_mldsa_cert.pem \
    -validity 3650 \
    -sign_alg mldsa65
```

Add the root CA and signed certificate to the server wallet:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet add \
    -wallet /u01/app/oracle/wallets/pqc-server \
    -trusted_cert \
    -cert /u01/app/oracle/wallets/pqc-root/pqc_root_ca_cert.pem

env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet add \
    -wallet /u01/app/oracle/wallets/pqc-server \
    -user_cert \
    -cert /u01/app/oracle/wallets/pqc-server/server_mldsa_cert.pem
```

Verify:

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet display \
    -wallet /u01/app/oracle/wallets/pqc-server \
    -complete
```

Expected:

```
User certificate:    CN=dbs19c.sub0409XXXXXX250.dbsec.oraclevcn.com
Trusted certificate: CN=PQC Root CA
```

6. Create the one-way TLS client trust wallet

For one-way TLS, the client requires only the root CA certificate. It does not need a client private key or client certificate.

```
env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet create \
    -wallet /u01/app/oracle/wallets/pqc-client \
    -auto_login

env -u ORACLE_HOME -u ORAPKI_JVM_ARGS \
  JAVA_HOME=/u01/app/oracle/product/jdk-24 \
  PATH=/u01/app/oracle/product/jdk-24/bin:$PATH \
  /u01/app/oracle/product/19.0.0/dbhome_1/bin/orapki wallet add \
    -wallet /u01/app/oracle/wallets/pqc-client \
    -trusted_cert \
    -cert /u01/app/oracle/wallets/pqc-root/pqc_root_ca_cert.pem
```

7. Deploy the server wallet to the database and listener

7.1 Determine `WALLET_ROOT` and PDB GUID

```
sqlplus "/as sysdba"
show parameter wallet_root

col name format a16
col guid format a32
select name, guid from v$pdbs where name = 'DBS19CPDB';
```

For the tested database:

```
WALLET_ROOT=/opt/oracle/dcs/commonstore/wallets/DB0730_329_lhr
PDB GUID=5691XXXXXXXXXXXXX74B8
```

The PDB TLS location is therefore:

```
/opt/oracle/dcs/commonstore/wallets/DB0730_329_lhr/5691XXXXXXXXXXXXXXX74B8/tls
```

Create the new PDB TLS directory and copy the runtime wallet files only. PEM files are not required there because the certificate chain is stored in `ewallet.p12` and `cwallet.sso`.

```
PQC_PDB_TLS=/opt/oracle/dcs/commonstore/wallets/DB0730_329_lhr/5691XXXXXXXXXXXXXXX74B8/tls

mkdir -p "$PQC_PDB_TLS"
chmod 700 "$PQC_PDB_TLS"
cp -p /u01/app/oracle/wallets/pqc-server/ewallet.p12 "$PQC_PDB_TLS/"
cp -p /u01/app/oracle/wallets/pqc-server/cwallet.sso "$PQC_PDB_TLS/"
chmod 600 "$PQC_PDB_TLS/ewallet.p12" "$PQC_PDB_TLS/cwallet.sso"
```

7.2 Update the listener and Oracle Net configuration

Back up the two files first:

```
export ORACLE_HOME=/u01/app/oracle/product/19.0.0/dbhome_1

cp -p $ORACLE_HOME/network/admin/listener.ora \
  $ORACLE_HOME/network/admin/listener.ora.pre_pqc
cp -p $ORACLE_HOME/network/admin/sqlnet.ora \
  $ORACLE_HOME/network/admin/sqlnet.ora.pre_pqc
```

The listener wallet and the PDB TLS wallet contain copies of the same server certificate, private key, and trust chain. The database server uses `WALLET_ROOT/<PDB_GUID>/tls`.

In `$ORACLE_HOME/network/admin/listener.ora`, configure the listener wallet location as follows:

```
WALLET_LOCATION =
  (SOURCE =
    (METHOD = FILE)
    (METHOD_DATA =
      (DIRECTORY = /u01/app/oracle/wallets/pqc-server)
    )
  )
```

Do not change this TDE line in `sqlnet.ora`:

```
ENCRYPTION_WALLET_LOCATION=(SOURCE=(METHOD=FILE)(METHOD_DATA=(DIRECTORY=.../tde)))
```

Use the TLS-prefixed parameters.

```
TLS_CLIENT_AUTHENTICATION=FALSE
TLS_VERSION=(1.3)
TLS_CIPHER_SUITES=(TLS_AES_256_GCM_SHA384,TLS_AES_128_GCM_SHA256)
TLS_KEY_EXCHANGE_GROUPS=(hybrid,ml-kem)
```

Avoid duplicate or partial `WALLET_LOCATION` stanzas. Remove or comment an entire old stanza, not just its first line. `SSL_` parameters are deprecated in favor of `TLS_` parameters.

Restart the listener and database during the test maintenance window:

```
$ORACLE_HOME/bin/lsnrctl stop LISTENER
$ORACLE_HOME/bin/lsnrctl start LISTENER
$ORACLE_HOME/bin/lsnrctl status LISTENER

sqlplus "/as sysdba"
shutdown immediate
startup
alter system register;
exit
```

Confirm the PDB service is `READY` in `lsnrctl status`.

8. Transfer and configure the client trust wallet

Copy only these files to the client host using a protected transfer method:

```
/u01/app/oracle/wallets/pqc-client/cwallet.sso
/u01/app/oracle/wallets/pqc-client/ewallet.p12
```

Never copy these directories to a client:

```
/u01/app/oracle/wallets/pqc-root     # contains the root CA private key
/u01/app/oracle/wallets/pqc-server   # contains the server private key
```

Example destination used in the test client:

```
/home/opc/dbs19c_pqc-client
```

Protect it:

```
mkdir -p /home/opc/dbs19c_pqc-client
chmod 700 /home/opc/dbs19c_pqc-client
chmod 600 /home/opc/dbs19c_pqc-client/ewallet.p12 \
          /home/opc/dbs19c_pqc-client/cwallet.sso
```

On the client, configure a single valid `WALLET_LOCATION` in the client `sqlnet.ora`:

```
WALLET_LOCATION =
  (SOURCE =
    (METHOD = FILE)
    (METHOD_DATA =
      (DIRECTORY = /home/opc/dbs19c_pqc-client)
    )
  )

SSL_SERVER_DN_MATCH=YES
TLS_VERSION=(1.3)
TLS_CLIENT_AUTHENTICATION=FALSE
TLS_CIPHER_SUITES=(TLS_AES_256_GCM_SHA384,TLS_AES_128_GCM_SHA256)
TLS_KEY_EXCHANGE_GROUPS=(hybrid,ml-kem)
```

9. Test from the client

The test connection that succeeded was:

```
sqlplus 'user@(
  DESCRIPTION=
    (ADDRESS=(PROTOCOL=TCPS)(HOST=dbs19c.sub0409XXXXXX250.dbsec.oraclevcn.com)(PORT=2484))
    (CONNECT_DATA=(SERVICE_NAME=dbs19cpdb.sub0409XXXXXX250.dbsec.oraclevcn.com))
    (SECURITY=(SSL_SERVER_DN_MATCH=YES))
)'
```

Use a valid user for the target PDB. SQL*Plus prompts for the password.

Expected outcome:

```
Connected to:
Oracle Database 19c ...
```

Confirm that the connection used TCPS:

```
select sys_context('USERENV','NETWORK_PROTOCOL') as network_protocol
from dual;
```

Expected:

```
tcps
```

The test client connected successfully using the PQC-root trust wallet and server DN matching.

10. Rollback

If the listener or applications fail after the PQC change:

```
cp -p $ORACLE_HOME/network/admin/listener.ora.pre_pqc \
  $ORACLE_HOME/network/admin/listener.ora
cp -p $ORACLE_HOME/network/admin/sqlnet.ora.pre_pqc \
  $ORACLE_HOME/network/admin/sqlnet.ora

$ORACLE_HOME/bin/lsnrctl stop LISTENER
$ORACLE_HOME/bin/lsnrctl start LISTENER
```

If the database must also return to its prior network configuration, restart it during the approved test window. Do not remove or overwrite TDE wallets.

References

- Oracle: Configuring Post-Quantum Cryptography — https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/config-post-quantum-crypto.html
- Oracle: Switching the Cryptographic Provider for Oracle Database 19c — https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/switching-crypto-providers.html
- Oracle: TLS wallet configuration — https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/about-configuring-tls-oracle-database.html
- Oracle: TLS protocol and cipher configuration — https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/advanced-and-optional-configurations-tls.html